

DAY 15: DATA VISUALIZATION WITH MATPLOTLIB

60-Day Data Science Learning Journey | Muhammad Ahmad

Prerequisite: Python, NumPy, Pandas and Data Cleaning.

Goal: Learn to convert data into meaningful visual insights with Matplotlib and Pandas.

1. What is Data Visualization?

Data Visualization represents data visually through charts and graphs. It helps us see trends, comparisons, distributions, relationships and unusual values.

```
import matplotlib.pyplot as plt
```

2. Why Visualization Matters

A Data Scientist must understand and communicate data. Visualizations often reveal patterns faster than raw tables and help during exploratory data analysis.

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [120, 150, 180, 130, 200]

plt.plot(months, sales)
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid()
plt.show()
```

3. Importing Matplotlib

Matplotlib is a Python visualization library. The pyplot interface is commonly imported as plt.

```
students = ["Ali", "Usman", "Hamza", "Ahmad"]
marks = [85, 72, 91, 78]

plt.bar(students, marks)
plt.title("Student Marks")
plt.xlabel("Student")
plt.ylabel("Marks")
plt.show()
```

4. Line Charts

Line charts are useful for trends, especially values changing over time.

```
plt.barh(students, marks)
plt.title("Student Marks")
plt.xlabel("Marks")
plt.ylabel("Student")
plt.show()
```

5. Bar Charts

Bar charts compare categories such as students, products or departments.

```
hours = [1, 2, 3, 4, 5, 6]
marks = [45, 50, 58, 65, 75, 85]

plt.scatter(hours, marks)
plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.show()
```

6. Horizontal Bar Charts

plt.barh() creates horizontal bars and can be useful when category labels are long.

```
marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 85, 90, 95]

plt.hist(marks, bins=5)
plt.title("Marks Distribution")
```

```
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.show()
```

7. Scatter Plots

Scatter plots show the relationship between two numerical variables, such as study hours and marks.

```
departments = ["CS", "SE", "AI"]
students = [50, 30, 20]

plt.pie(
    students,
    labels=departments,
    autopct="%1.1f%%"
)
plt.title("Students by Department")
plt.show()
```

8. Histograms

Histograms show the distribution of numerical data. bins controls how values are grouped.

```
plt.title("Daily Sales")
plt.xlabel("Day")
plt.ylabel("Sales")
plt.grid()
```

9. Pie Charts

Pie charts show parts of a whole. labels identify slices and autopct can display percentages.

```
months = ["Jan", "Feb", "Mar", "Apr"]
sales_2025 = [100, 120, 150, 170]
sales_2026 = [110, 140, 160, 200]

plt.plot(months, sales_2025, label="2025")
plt.plot(months, sales_2026, label="2026")
plt.legend()
plt.show()
```

10. Titles, Labels and Grid

Titles and axis labels explain a chart. A grid can make values easier to read.

```
plt.figure(figsize=(10, 6))
plt.xticks(rotation=45)
```

11. Legends

Legends identify multiple data series. Use label= when plotting and plt.legend() to display the legend.

```
plt.subplot(1, 2, 1)
plt.bar(students, marks)
plt.title("Bar Chart")

plt.subplot(1, 2, 2)
plt.plot(students, marks)
plt.title("Line Chart")

plt.show()
```

12. Figure Size and X-Ticks

figsize controls chart dimensions. plt.xticks(rotation=45) can improve readability of long category labels.

```
plt.savefig("student_marks.png")
```

13. Multiple Charts

Subplots allow several charts in one figure. plt.subplot() is a basic method, while fig, ax = plt.subplots() is a flexible modern approach.

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
data = {  
    "Name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal"],  
    "Marks": [85, 72, 91, 78, 65],  
    "CGPA": [3.2, 3.0, 3.6, 3.1, 2.8]  
}  
  
df = pd.DataFrame(data)  
  
plt.bar(df["Name"], df["Marks"])  
plt.title("Student Marks")  
plt.show()
```

16. Complete Student Performance Visualization Project

This project combines Pandas analysis with four visualizations: marks, CGPA, study hours versus marks, and marks distribution.

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal", "Hassan"],
    "Marks": [85, 72, 91, 78, 65, 88],
    "CGPA": [3.2, 3.0, 3.6, 3.1, 2.8, 3.4],
    "Study_Hours": [5, 4, 7, 5, 3, 6]
}

df = pd.DataFrame(data)

average_marks = df["Marks"].mean()
average_cgpa = df["CGPA"].mean()
highest_marks = df["Marks"].max()
top_student = df.loc[df["Marks"].idxmax()]

print("Average Marks:", average_marks)
print("Average CGPA:", average_cgpa)
print("Highest Marks:", highest_marks)
print("\nTop Student:")
print(top_student)

plt.figure(figsize=(10, 6))
plt.bar(df["Name"], df["Marks"])
plt.title("Student Marks")
plt.xlabel("Student")
plt.ylabel("Marks")
plt.grid(axis="y")
plt.show()

plt.figure(figsize=(10, 6))
plt.bar(df["Name"], df["CGPA"])
plt.title("Student CGPA")
plt.xlabel("Student")
plt.ylabel("CGPA")
plt.grid(axis="y")
plt.show()

plt.figure(figsize=(10, 6))
plt.scatter(df["Study_Hours"], df["Marks"])
plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.grid()
plt.show()

plt.figure(figsize=(10, 6))
plt.hist(df["Marks"], bins=5)
plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.grid()
plt.show()
```

Expected analysis: average marks = 79.8333, average CGPA = 3.1833, highest marks = 91, and Hamza is the top student by marks.

Interpretation: A scatter plot can reveal an apparent relationship between study hours and marks, but visualization alone does not prove causation.

17. Practice Tasks

Task 1: Create a line chart for Jan-May sales [100, 150, 120, 180, 220] with title, axis labels and grid.

Task 2: Create a bar chart for Ali=80, Usman=75, Hamza=95 and Ahmad=88.

Task 3: Create a scatter plot with Study Hours [2,3,4,5,6,7] and Marks [50,55,65,70,80,90].

Task 4: Create a histogram from [45,50,55,60,62,65,68,70,72,75,80,85,90,95] using bins=5.

Task 5: Create a pie chart for CS=50, SE=30 and AI=20 and show percentages.

Task 6: Create a Pandas DataFrame with Name, Marks and CGPA for at least five students and visualize both Marks and CGPA.

Task 7: Create a Marks vs CGPA scatter plot.

Task 8: Create a marks histogram.

Task 9: Save a marks chart as student_marks.png.

Task 10: Build a Student Performance Dashboard with marks bar chart, CGPA bar chart, Study Hours vs Marks scatter plot and marks histogram.

18. Complete Task Solutions

Solution 1

```
import matplotlib.pyplot as plt
months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [100, 150, 120, 180, 220]
plt.plot(months, sales)
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid()
plt.show()
```

Solution 2

```
students = ["Ali", "Usman", "Hamza", "Ahmad"]
marks = [80, 75, 95, 88]
plt.bar(students, marks)
plt.title("Student Marks")
plt.show()
```

Solution 3

```
hours = [2, 3, 4, 5, 6, 7]
marks = [50, 55, 65, 70, 80, 90]
plt.scatter(hours, marks)
plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.show()
```

Solution 4

```
marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 80, 85, 90, 95]
plt.hist(marks, bins=5)
plt.title("Marks Distribution")
plt.show()
```

Solution 5

```
departments = ["CS", "SE", "AI"]
students = [50, 30, 20]
plt.pie(students, labels=departments, autopct="%1.1f%%")
plt.title("Students by Department")
plt.show()
```

Solution 6

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal"],
    "Marks": [85, 72, 91, 78, 65],
    "CGPA": [3.2, 3.0, 3.6, 3.1, 2.8]
}
df = pd.DataFrame(data)
plt.bar(df["Name"], df["Marks"])
plt.show()
plt.bar(df["Name"], df["CGPA"])
plt.show()
```

Solution 7

```
plt.scatter(df["Marks"], df["CGPA"])
plt.title("Marks vs CGPA")
plt.xlabel("Marks")
plt.ylabel("CGPA")
```

```
plt.show()
```

Solution 8

```
plt.hist(df["Marks"], bins=5)
plt.title("Marks Distribution")
plt.show()
```

Solution 9

```
plt.bar(df["Name"], df["Marks"])
plt.title("Student Marks")
plt.savefig("student_marks.png")
plt.show()
```

Solution 10

```
# Use the complete project above.
# It creates the four requested visualizations.
```

19. Common Mistakes

- Using x and y lists with different lengths.
- Forgetting plt.show() in a normal script.
- Choosing a chart that does not match the question.
- Creating charts without clear titles or axis labels.
- Putting too many categories into a pie chart.
- Treating visual association as proof of causation.
- Saving a chart before finishing its labels and formatting.

20. Quick Revision

Chart / Tool	Purpose
plt.plot()	Trends over an ordered variable or time
plt.bar()	Category comparison
plt.barh()	Horizontal category comparison
plt.scatter()	Relationship between two numerical variables
plt.hist()	Distribution of numerical values
plt.pie()	Parts of a whole
plt.title()	Chart title
plt.xlabel() / plt.ylabel()	Axis labels
plt.grid()	Reference grid
plt.legend()	Identify multiple series
plt.figure(figsize=(10,6))	Control figure size
plt.savefig()	Save chart
plt.show()	Display chart

21. Final Checklist

- I understand Data Visualization.
- I can import Matplotlib.
- I can create line, bar, scatter, histogram and pie charts.
- I can add titles, labels, grids and legends.
- I can control figure size and rotate x labels.

- I can create multiple charts.
- I can save charts as image files.
- I can visualize Pandas DataFrames.
- I can interpret charts without confusing association with causation.
- I completed the Student Performance Visualization Project.

22. Final Takeaway

Day 15 connects your Pandas and Data Cleaning knowledge with visual analysis. Visualization is not only about making attractive charts. It is about asking useful questions, seeing patterns, communicating findings and preparing for deeper statistical analysis and Machine Learning.

Learning path: Python → NumPy → Pandas → Data Cleaning → Data Visualization → Statistics → Machine Learning → Deep Learning